PES1UG24AM240_SE-LAB-7_REPORT

Name:Sairam Charan N
SRN:PES1UG24AM240

CLASS:CSE(AiML)/5-E

1. Objective

The objective of this lab is to apply fuzz testing to a Python application, use the Hypothesis property-based testing library to generate unexpected and malformed inputs, identify input-validation bugs, and improve the robustness of the application through defensive programming.

2. Tools and Environment

Component	Details
Programming language	Python 3.12.6
Testing framework	pytest 8.3.4
Fuzzing library	Hypothesis 6.170.15
Operating system	Windows (win32)
Application under test	processor.py
Test module	test_processor.py

3. Functions Tested

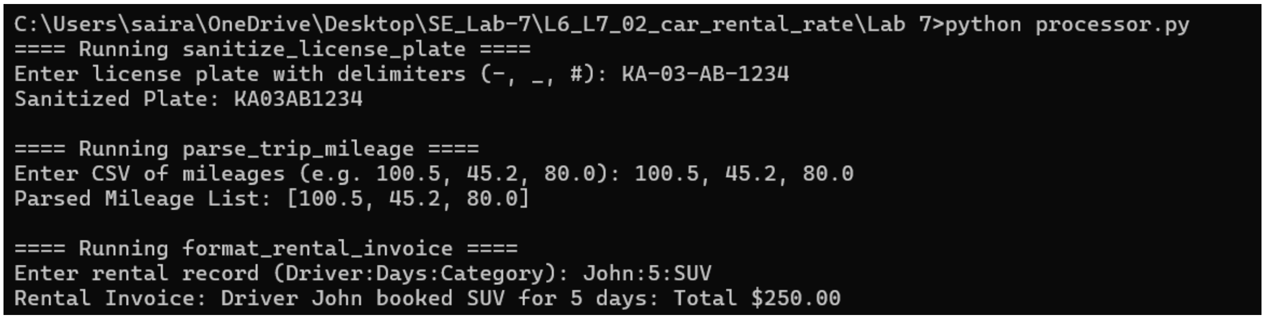
- **sanitize_license_plate()** — Removes supported delimiters from a license plate and converts the result to uppercase.
- **parse_trip_mileage()** — Parses comma-separated mileage values into a list of numeric values.
- **format_rental_invoice()** — Parses a Driver:Days:Category rental record and formats the rental total.

4. Testing Procedure and Findings

4.1 Original application execution

The original application was executed with representative valid inputs. The three functions completed and produced the expected normal-case outputs: sanitized plate KA03AB1234, mileage list [100.5, 45.2, 80.0], and an SUV rental invoice total of $250.00.

PES1UG24AM240_SE-LAB-7_REPORT

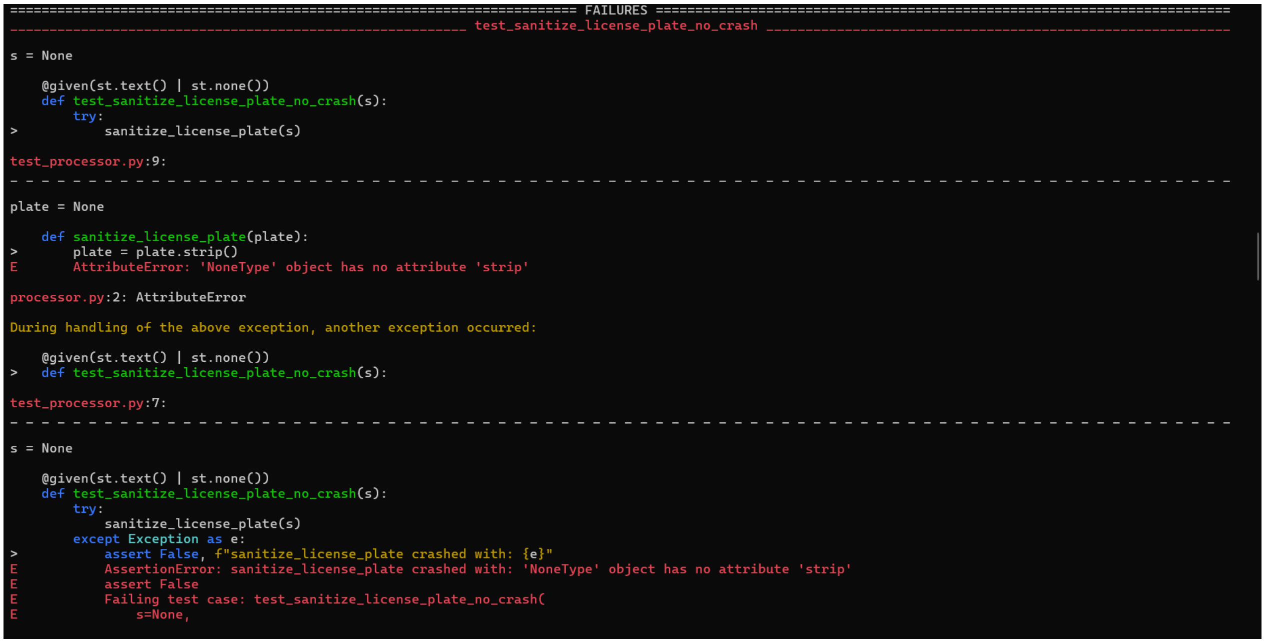


```
C:\Users\saira\OneDrive\Desktop\SE_Lab-7\L6_L7_02_car_rental_rate\Lab 7>python processor.py
==== Running sanitize_license_plate ====
Enter license plate with delimiters (-, _, #): KA-03-AB-1234
Sanitized Plate: KA03AB1234

==== Running parse_trip_mileage ====
Enter CSV of mileages (e.g. 100.5, 45.2, 80.0): 100.5, 45.2, 80.0
Parsed Mileage List: [100.5, 45.2, 80.0]

==== Running format_rental_invoice ====
Enter rental record (Driver:Days:Category): John:5:SUV
Rental Invoice: Driver John booked SUV for 5 days: Total $250.00
```

Figure 1. Original application running with valid sample inputs.

4.2 Hypothesis-based fuzz testing

Three property-based tests were added using @given(st.text() | st.none()). Each test invokes one application function and reports an assertion failure if an exception is raised. The initial run collected three tests and all three failed, demonstrating that unexpected inputs could crash the original implementation.



```
======================================================================= FAILURES ========================================================================
_________________________________________________________ test_sanitize_license_plate_no_crash __________________________________________________________

s = None

    @given(st.text() | st.none())
    def test_sanitize_license_plate_no_crash(s):
        try:
>           sanitize_license_plate(s)

test_processor.py:9:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

plate = None

    def sanitize_license_plate(plate):
>       plate = plate.strip()
E       AttributeError: 'NoneType' object has no attribute 'strip'

processor.py:2: AttributeError

During handling of the above exception, another exception occurred:

    @given(st.text() | st.none())
>   def test_sanitize_license_plate_no_crash(s):

test_processor.py:7:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

s = None

    @given(st.text() | st.none())
    def test_sanitize_license_plate_no_crash(s):
        try:
            sanitize_license_plate(s)
        except Exception as e:
>           assert False, f"sanitize_license_plate crashed with: {e}"
E           AssertionError: sanitize_license_plate crashed with: 'NoneType' object has no attribute 'strip'
E           assert False
E           Failing test case: test_sanitize_license_plate_no_crash(
E               s=None,
```

Figure 2. Hypothesis exposes an input-validation failure in the original application.

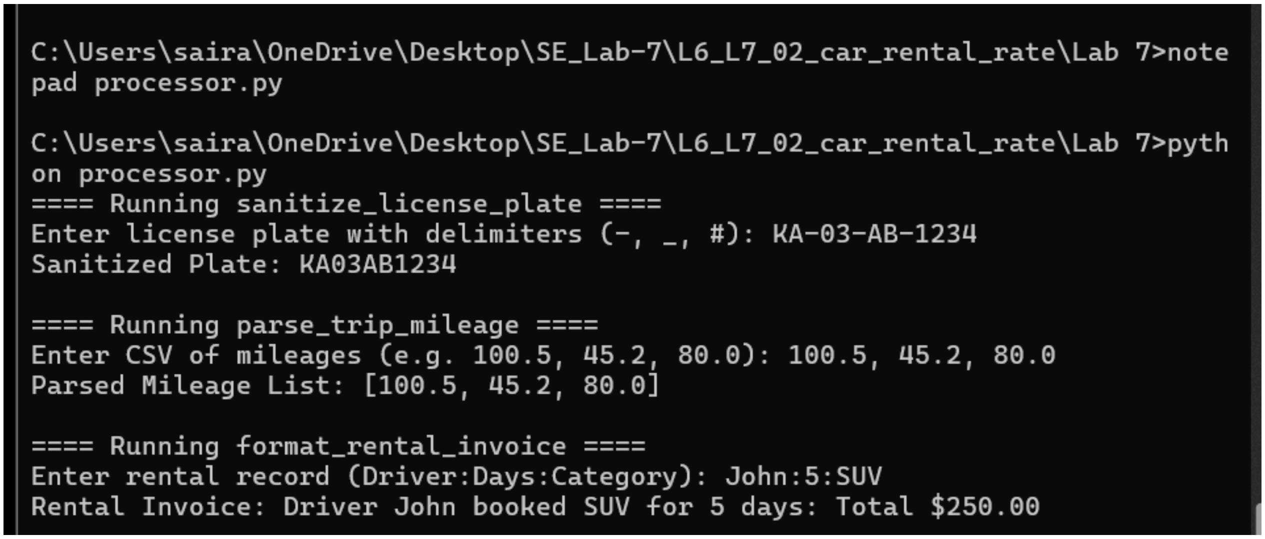
The reported failures included the following minimal or representative inputs:

- sanitize_license_plate(None): AttributeError because None has no strip() method.
- parse_trip_mileage(''): ValueError when an empty element is converted to float.
- parse_trip_mileage(None): AttributeError because None has no split() method.
- format_rental_invoice(':'): ValueError because the days field is empty.
- format_rental_invoice(''): IndexError because required record fields are missing.

- format_rental_invoice(None): AttributeError because None has no split() method.

4.3 Defensive fixes applied

The application was updated to validate input types and structure before processing. The license-plate function now handles non-string values safely. The mileage parser handles non-string and empty input, trims individual values, skips invalid numeric entries, and excludes negative mileage values. The invoice formatter checks the record type, requires exactly three fields, validates the day count, and rejects missing driver/category values and non-positive rental durations. Valid-input formatting and the original rate rule ($50/day for SUV; $30/day for other categories) were retained.

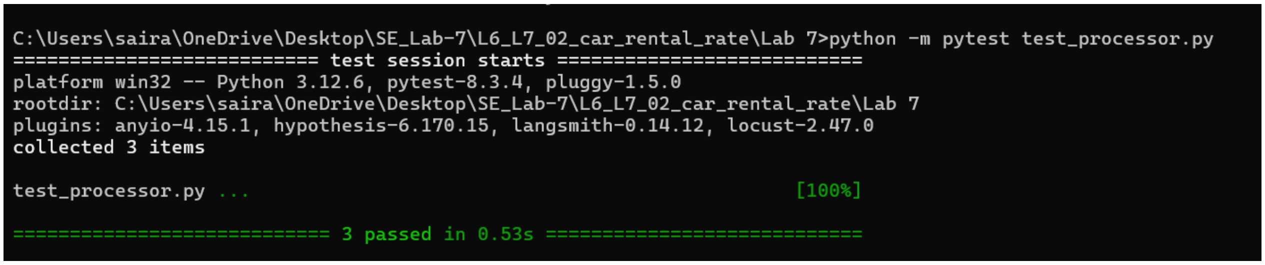


```
C:\Users\saira\OneDrive\Desktop\SE_Lab-7\L6_L7_02_car_rental_rate\Lab 7>note
pad processor.py

C:\Users\saira\OneDrive\Desktop\SE_Lab-7\L6_L7_02_car_rental_rate\Lab 7>pyth
on processor.py
==== Running sanitize_license_plate ====
Enter license plate with delimiters (-, _, #): KA-03-AB-1234
Sanitized Plate: KA03AB1234

==== Running parse_trip_mileage ====
Enter CSV of mileages (e.g. 100.5, 45.2, 80.0): 100.5, 45.2, 80.0
Parsed Mileage List: [100.5, 45.2, 80.0]

==== Running format_rental_invoice ====
Enter rental record (Driver:Days:Category): John:5:SUV
Rental Invoice: Driver John booked SUV for 5 days: Total $250.00
```

Figure 3. Fixed application executing successfully with the same valid sample inputs.

4.4 Final fuzz-test verification

The same Hypothesis test module was executed again against the fixed application. Pytest collected three tests and reported that all three passed in 0.53 seconds.



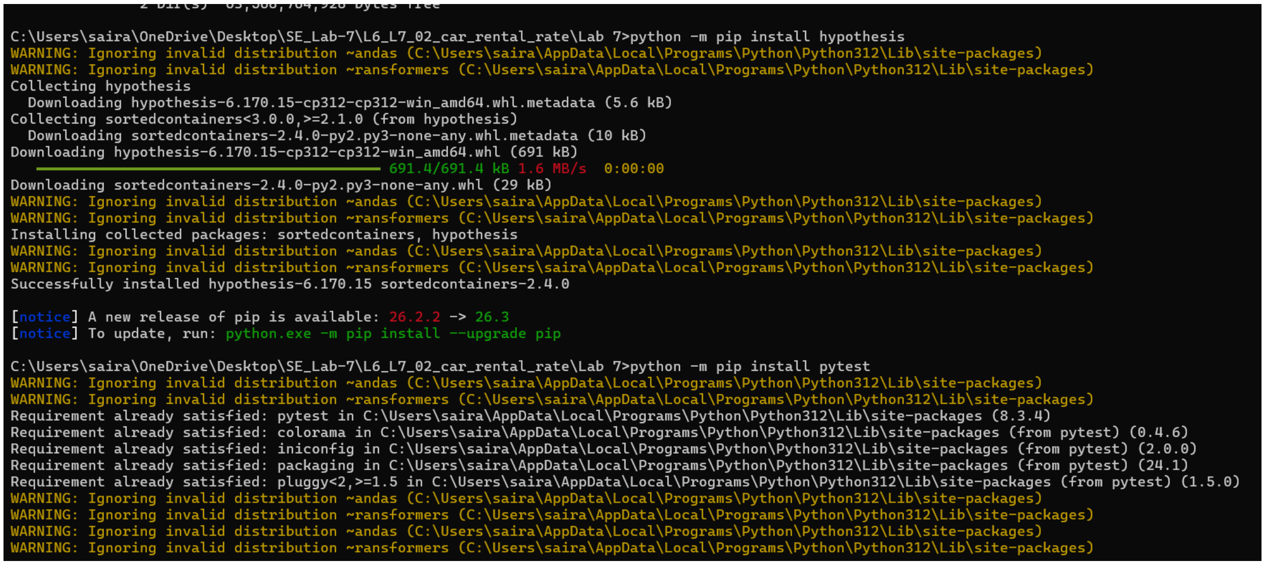
```
C:\Users\saira\OneDrive\Desktop\SE_Lab-7\L6_L7_02_car_rental_rate\Lab 7>python -m pytest test_processor.py
=========================== test session starts ===========================
platform win32 -- Python 3.12.6, pytest-8.3.4, pluggy-1.5.0
rootdir: C:\Users\saira\OneDrive\Desktop\SE_Lab-7\L6_L7_02_car_rental_rate\Lab 7
plugins: anyio-4.15.1, hypothesis-6.170.15, langsmith-0.14.12, locust-2.47.0
collected 3 items

test_processor.py ...                                                [100%]

============================ 3 passed in 0.53s ============================
```

Figure 4. Final pytest result: 3 passed.

PES1UG24AM240_SE-LAB-7_REPORT

5. Setup Evidence

The dependencies required by the handout were installed or verified in the Windows command prompt: Hypothesis and pytest.



```
C:\Users\saira\OneDrive\Desktop\SE_Lab-7\L6_L7_02_car_rental_rate\Lab 7>python -m pip install hypothesis
WARNING: Ignoring invalid distribution ~andas (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ransformers (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
Collecting hypothesis
  Downloading hypothesis-6.170.15-cp312-cp312-win_amd64.whl.metadata (5.6 kB)
Collecting sortedcontainers<3.0.0,>=2.1.0 (from hypothesis)
  Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Downloading hypothesis-6.170.15-cp312-cp312-win_amd64.whl (691 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 691.4/691.4 kB 1.6 MB/s  0:00:00
Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
WARNING: Ignoring invalid distribution ~andas (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ransformers (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
Installing collected packages: sortedcontainers, hypothesis
WARNING: Ignoring invalid distribution ~andas (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ransformers (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
Successfully installed hypothesis-6.170.15 sortedcontainers-2.4.0

[notice] A new release of pip is available: 26.2.2 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\saira\OneDrive\Desktop\SE_Lab-7\L6_L7_02_car_rental_rate\Lab 7>python -m pip install pytest
WARNING: Ignoring invalid distribution ~andas (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ransformers (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
Requirement already satisfied: pytest in C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages (8.3.4)
Requirement already satisfied: colorama in C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig in C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pytest) (2.0.0)
Requirement already satisfied: packaging in C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pytest) (24.1)
Requirement already satisfied: pluggy<2,>=1.5 in C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pytest) (1.5.0)
WARNING: Ignoring invalid distribution ~andas (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ransformers (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~andas (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ransformers (C:\Users\saira\AppData\Local\Programs\Python\Python312\Lib\site-packages)
```

Figure 5. Dependency installation and verification.

6. Conclusion

This experiment demonstrated how property-based fuzz testing can reveal defects that are not exposed by ordinary valid-input execution. Hypothesis discovered failures involving None values, empty strings, malformed CSV values, and incomplete rental records. Defensive validation was then added to all three functions. Re-running the same test suite resulted in 3 passed tests, confirming that the tested unexpected-input cases no longer caused uncaught exceptions.